

SEL0615 – Processamento Digital de Sinais

Exercício 2 – Implementação da DFT e Análise Espectral

Nome: Bárbara Fernandes
Nº USP: 11915032

17 de março de 2026

1 Introdução

Neste exercício, foi desenvolvida uma implementação da DFT em Python, a partir do script apresentado em aula, complementando-o com rotinas para leitura de arquivos de entrada, cálculo da frequência de amostragem e detecção das componentes harmônicas presentes no sinal analisado.

2 Objetivo

O objetivo deste trabalho é ler o sinal armazenado em `Sinal.txt` e sua estampa de tempo em `TimeStamp.txt`, determinar a frequência de amostragem do sinal, calcular a Transformada Discreta de Fourier (DFT), obter o espectro unilateral de amplitudes e identificar a componente fundamental e os harmônicos presentes.

3 Fundamentação teórica

A DFT de um sinal discreto $x[n]$, com N amostras, é dada por:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j \frac{2\pi}{N} kn}, \quad k = 0, 1, \dots, N-1 \quad (1)$$

onde:

- $x[n]$ representa o sinal no domínio do tempo;
- $X[k]$ representa o espectro no domínio da frequência;
- N é o número total de amostras.

A frequência associada a cada índice k da DFT é dada por:

$$f_k = \frac{k f_s}{N} \quad (2)$$

em que f_s é a frequência de amostragem. Essa frequência pode ser calculada a partir do intervalo médio entre amostras:

$$f_s = \frac{1}{\Delta t} \quad (3)$$

sendo Δt o espaçamento temporal entre duas amostras consecutivas.

Código em Python

O código a seguir realiza a leitura dos arquivos `Sinal.txt` e `TimeStamp.txt`, calcula a frequência de amostragem, implementa manualmente a Transformada Discreta de Fourier (DFT), gera o espectro unilateral e identifica a frequência fundamental e os harmônicos presentes no sinal.

```
1 import numpy as np
2 from pathlib import Path
3
4 # =====
5 # FUNÇÃO: ler_vetor
6 # Objetivo:
7 # Ler um arquivo .txt com n meros separados por ';'
8 # e transformar esse conteúdo em um vetor NumPy.
9 # =====
10 def ler_vetor(caminho_arquivo):
11     # Abre o arquivo em modo leitura
12     with open(caminho_arquivo, "r", encoding="utf-8") as arquivo:
13         conteudo = arquivo.read()
14
15     # Remove quebras de linha e espa os extras
16     conteudo = conteudo.replace("\n", "").replace("\r", "").strip()
17
18     # Divide o texto usando ';' como separador
19     # e converte cada elemento para float
20     valores = [float(v) for v in conteudo.split(";") if v.strip() != ""]
21
22     # Retorna os valores como um array NumPy
23     return np.array(valores, dtype=float)
24
25
26 # =====
27 # FUNÇÃO: dft
28 # Objetivo:
29 # Implementar manualmente a Transformada Discreta de
30 # Fourier (DFT), conforme a fórmula vista em aula.
31 #
32 # Entrada:
33 # x -> vetor do sinal no domínio do tempo
34 #
35 # Saída:
36 # X -> vetor complexo com o espectro em frequência
37 # =====
38 def dft(x):
39     # Número de amostras do sinal
40     N = len(x)
41
42     # Cria um vetor complexo para armazenar a DFT
```

```

43 X = np.zeros(N, dtype=complex)
44
45 # Loop externo: percorre cada ndice de frequencia k
46 for k in range(N):
47     # Inicializa a soma complexa da formula da DFT
48     soma = 0j
49
50     # Loop interno: percorre cada amostra do sinal no tempo
51     for n in range(N):
52         # Formula da DFT:
53         #  $X[k] = \text{soma de } x[n] * e^{(-j*2*\pi*k*n/N)}$ 
54         soma += x[n] * np.exp(-2j * np.pi * k * n / N)
55
56     # Armazena o valor calculado para a frequencia k
57     X[k] = soma
58
59 # Retorna o vetor da DFT
60 return X
61
62
63 # =====
64 # FUNÇÃO: espectro_unilateral
65 # A partir da DFT, gerar:
66 # - vetor de frequencias em Hz
67 # - vetor de amplitudes do espectro unilateral
68 #
69 # Entrada:
70 # X -> resultado da DFT
71 # fs -> frequencia de amostragem
72 #
73 # Saída:
74 # freq_pos -> frequencias positivas
75 # amp -> amplitudes associadas
76 # =====
77 def espectro_unilateral(X, fs):
78     # Numero total de pontos da DFT
79     N = len(X)
80
81     # Construi o vetor de frequencias correspondente a cada ndice da
82     # DFT
83     #  $f_k = k*fs/N$ 
84     freq = np.arange(N) * fs / N
85
86     # Caso N seja par
87     if N % 2 == 0:
88         # ndice da metade do espectro
89         metade = N // 2
90
91         # Mantem apenas a parte positiva do espectro, incluindo Nyquist
92         freq_pos = freq[:metade + 1]
93
94         # Calcula a amplitude normalizada
95         amp = np.abs(X[:metade + 1]) / N
96
97         # Como estamos usando espectro unilateral,
98         # dobramos as amplitudes internas
99         # (exceto DC e Nyquist)
100         amp[1:-1] *= 2

```

```

100
101     # Caso N seja mpar
102     else:
103         metade = N // 2
104
105         # Mant m apenas a parte positiva
106         freq_pos = freq[:metade + 1]
107
108         # Calcula a amplitude normalizada
109         amp = np.abs(X[:metade + 1]) / N
110
111         # Para N mpar , dobramos todas as amplitudes
112         # exceto a componente DC
113         amp[1:] *= 2
114
115         # Retorna frequencias positivas e amplitudes
116         return freq_pos, amp
117
118
119 # =====
120 # FUN 0: encontrar_harmonicos
121 # Encontrar picos no espectro que representem
122 # frequencias relevantes do sinal.
123 #
124 # Entrada:
125 #   freq          -> vetor de frequencias
126 #   amp           -> vetor de amplitudes
127 #   limiar_relativo -> percentual da maior amplitude
128 #                   usado para filtrar picos pequenos
129 #
130 # Saída:
131 #   indices dos picos encontrados
132 # =====
133 def encontrar_harmonicos(freq, amp, limiar_relativo=0.05):
134     # Se o espectro tiver menos de 3 pontos,
135     # n o h como verificar pico local
136     if len(amp) < 3:
137         return np.array([], dtype=int)
138
139     # Ignora a componente DC (0 Hz) ao calcular o limiar
140     amp_sem_dc = amp[1:] if len(amp) > 1 else amp
141
142     # Define o valor m nimo para um pico ser considerado relevante
143     limiar = limiar_relativo * np.max(amp_sem_dc)
144
145     # Lista para armazenar os indices dos picos
146     indices = []
147
148     # Percorre o vetor de amplitudes procurando m ximos locais
149     for i in range(1, len(amp) - 1):
150         # Um pico aceito se:
151         # 1) for maior que o vizinho anterior
152         # 2) for maior ou igual ao pr ximo
153         # 3) estiver acima do limiar m nimo
154         if amp[i] > amp[i - 1] and amp[i] >= amp[i + 1] and amp[i] >=
            limiar:
155             indices.append(i)
156

```

```

157     # Retorna os ndices encontrados como array NumPy
158     return np.array(indices, dtype=int)
159
160
161 # =====
162 # FUNÇÃO PRINCIPAL
163 # 1. Encontrar os arquivos na mesma pasta do script
164 # 2. Ler sinal e tempos
165 # 3. Calcular frequencia de amostragem
166 # 4. Calcular a DFT
167 # 5. Gerar o espectro unilateral
168 # 6. Detectar harmônicos
169 # 7. Exibir resultados
170 # =====
171 def main():
172     # Descobre automaticamente a pasta onde está este script Python
173     pasta = Path(__file__).resolve().parent
174
175     # Monta os caminhos completos dos arquivos de entrada
176     arquivo_sinal = pasta / "Sinal.txt"
177     arquivo_tempo = pasta / "TimeStamp.txt"
178
179     # Lê os dados dos arquivos
180     sinal = ler_vetor(arquivo_sinal)
181     tempo = ler_vetor(arquivo_tempo)
182
183     # Verifica se os dois vetores têm o mesmo tamanho
184     # Cada amostra do sinal deve ter um instante correspondente
185     if len(sinal) != len(tempo):
186         raise ValueError(
187             f"O número de amostras é diferente: sinal={len(sinal)} e "
188             f"tempo={len(tempo)}"
189         )
190
191     # Número total de amostras
192     N = len(sinal)
193
194     # Calcula a diferença entre instantes consecutivos
195     dt = np.diff(tempo)
196
197     # Calcula o passo médio de amostragem
198     dt_medio = np.mean(dt)
199
200     # Frequência de amostragem: fs = 1 / dt
201     fs = 1 / dt_medio
202
203     # -----
204     # Exibir o início dos dados
205     # -----
206     print("=" * 50)
207     print("LEITURA DOS ARQUIVOS")
208     print("=" * 50)
209     print(f"Arquivo do sinal: {arquivo_sinal}")
210     print(f"Arquivo do tempo: {arquivo_tempo}")
211     print(f"Quantidade de amostras: {N}")
212     print(f"Primeiras 5 amostras do sinal: {sinal[:5]}")
213     print(f"Primeiros 5 instantes: {tempo[:5]}")

```

```

214 # -----
215 # Exibi o da frequencia de amostragem
216 # -----
217 print("\n" + "=" * 50)
218 print("FREQU NCIA DE AMOSTRAGEM")
219 print("=" * 50)
220 print(f"dt m dio = {dt_medio:.12f} s")
221 print(f"fs = {fs:.6f} Hz")
222
223 # Calcula a DFT do sinal
224 X = dft(sinal)
225
226 # Gera o espectro unilateral
227 freq, amp = espectro_unilateral(X, fs)
228
229 # Procura picos no espectro
230 indices_harmonicos = encontrar_harmonicos(freq, amp,
231     limiar_relativo=0.05)
232
233 # Remove a componente em 0 Hz, se aparecer
234 indices_harmonicos = indices_harmonicos[freq[indices_harmonicos] > 0]
235
236 # -----
237 # Exibi o dos harm nicos encontrados
238 # -----
239 print("\n" + "=" * 50)
240 print("HARM NICOS ENCONTRADOS")
241 print("=" * 50)
242
243 # Se nenhum pico foi encontrado, encerra
244 if len(indices_harmonicos) == 0:
245     print("Nenhum harm nico encontrado com o limiar atual.")
246     return
247
248 # A frequencia fundamental o menor pico positivo encontrado
249 indice_fundamental =
250     indices_harmonicos[np.argmin(freq[indices_harmonicos])]
251
252 # Frequencia e amplitude da fundamental
253 freq_fundamental = freq[indice_fundamental]
254 amp_fundamental = amp[indice_fundamental]
255
256 # Mostra a componente fundamental
257 print(f"Frequencia fundamental: {freq_fundamental:.6f} Hz")
258 print(f"Amplitude fundamental: {amp_fundamental:.6f}\n")
259
260 # Mostra todos os harm nicos detectados
261 for idx in indices_harmonicos:
262     print(f"Frequencia = {freq[idx]:.6f} Hz | Amplitude =
263         {amp[idx]:.6f}")
264
265 # =====
266 # Ponto de entrada do programa
267 # O c digo dentro de main() s ser executado se este
268 # arquivo for rodado diretamente.
269 # =====
270 if __name__ == "__main__":

```

Listing 1: Código Python comentado para análise espectral

3.1 Determinação das frequências harmônicas e de suas amplitudes

Com base na execução do programa, verificou-se que a frequência fundamental do sinal é 60 Hz, com amplitude igual a 129,9. Além disso, foi identificado um harmônico em 180 Hz, cuja amplitude é 11,4. Portanto, considerando o critério de detecção adotado no código, conclui-se que o sinal é composto pela componente fundamental de 60 Hz e por um harmônico em 180 Hz.

Frequência (Hz)	Amplitude
60	129,9
180	11,4

4 Conclusão

Conclui-se que a implementação da Transformada Discreta de Fourier em Python permitiu analisar corretamente o sinal fornecido, determinando sua frequência de amostragem como 3840 Hz a partir dos dados de `TimeStamp.txt`. A análise espectral identificou uma componente fundamental em 60 Hz, com amplitude de 129,9, e um harmônico em 180 Hz, com amplitude de 11,4. Dessa forma, os resultados mostram que o sinal é composto predominantemente pela frequência de 60 Hz, com presença de uma componente harmônica de menor amplitude em 180 Hz, evidenciando a utilidade da DFT na identificação das frequências presentes em sinais discretos.